

Relazione Tecnica Progetto #4 - Analisi Producibilità Eolica

A cura di:

Francesco Angelo D'Onghia (0001134261)

Nicolò Giaculli (0001089907)

Matteo Pierantozzi (0001129788)

Francesco Laurenti (0001123177)

1. Introduzione

Il programma parte con dei dati valori di α e β parametri di una **distribuzione Weibull**, il programma mira ad utilizzare un montecarlo per determinare il numero minimo di dati necessari a stimare un valore corretto di α e β , il numero di iterazioni del montecarlo viene deciso inizialmente come **input** della **function**.

Nello specifico, partendo da valori noti $\alpha = 19 \text{ m/s}$ e $\beta = 3.1$, si intende determinare il numero minimo di campioni (n) necessario per stimare tali parametri con un errore inferiore al **15%**.

2. Cenni Teorici

La **distribuzione di Weibull** è definita dalla funzione di distribuzione cumulativa:

$$F(v) = 1 - e^{-\left(\frac{v}{\alpha}\right)^\beta}$$

Per stimare i parametri a partire da un set di dati sperimentali, si è proceduto alla linearizzazione della funzione, applicando due volte il ln:

$$\beta \ln(x) - \beta \ln(\alpha) = \ln(-\ln(1 - F(x)))$$

Utilizzando la **carta probabilistica di Weibull**, l'equazione diventa del tipo $Y = mX + q$, dove:

- $X = \ln(x)$
- $Y = \ln(-\ln(1 - F(x)))$
- $m = \beta$
- $q = -\beta \ln(\alpha)$

La probabilità cumulata F è stata calcolata mediante lo **stimatore di Blom**:

$$F = \frac{i - 0.5}{n + 0.25}$$

dove i rappresenta la posizione del dato nel campione ordinato.

3. Architettura del Software

Il codice **MATLAB** è stato strutturato in uno **Script principale (Main)**, che lancia il programma, una **function primaria** (“esame”) e tre **function secondarie**, per garantire modularità e leggibilità.

3.1. Function Principale

Gestisce il **ciclo Montecarlo** per M iterazioni. Per ogni iterazione e per diverse numerosità del lotto ($n \in \{3, 5, 10, 20, 40\}$), richiama le funzioni di calcolo e immagazzina i risultati nei vettori α e β .

```
function [n_ottimale] = esame(alpha, beta, M)
% ESAME - Funzione principale per la simulazione Montecarlo della
% distribuzione di Weibull

% INPUT:
% alpha : Parametro di forma reale
% beta  : Parametro di scala reale
% M     : Numero di iterazioni Montecarlo
%
% OUTPUT:
%n_ottimale: dopo una verifica sui valori estratti dal montecarlo (devono avere uno
scarto percentuale minore o uguale al 15% rispetto ai valori reali) si seleziona il
valore con il minor numero di estrazioni che passa le suddette verifiche.
```

In questa prima parte si impostano tutti i valori di input e di output del programma:

- α è il parametro di scala della **Weibull** di partenza, ottenuto tramite un approccio di tipo frequenziale; quindi, avendo a disposizione un elevato numero di campioni inerenti alla velocità del vento.
- β è il parametro di forma della **Weibull** di partenza.
- M è il numero di cicli che deve attuare il montecarlo.

- **N_ottimale** è l'output della **function** questo preleva il minor numero di estrazioni che soddisfa il minimo scarto percentuale di α e β , pari al 15%.

Impostazione del problema e introduzioni delle variabili utili per l'esecuzione del programma

```
% Definisco le estrazioni che devo confrontare
n_estrazioni = [3, 5, 10, 20, 40];
%n_estrazioni = input(['inserire 'n' array del numero ' ...
%   'di estrazione variabile da 3 a 40: ']);
num_casistiche = length(n_estrazioni);

% Pre-allocazione delle matrici per velocità e gestione memoria
% È buona norma pre-allocare quando si lavora con M elevato
alpha_matrice = zeros(num_casistiche, M);
beta_matrice = zeros(num_casistiche, M);

% Vettori per le statistiche
atteso_Alpha = zeros(1, num_casistiche);
mediano_Alpha = zeros(1, num_casistiche);
dev_Alpha = zeros(1, num_casistiche);

atteso_Beta = zeros(1, num_casistiche);
mediano_Beta = zeros(1, num_casistiche);
dev_Beta = zeros(1, num_casistiche);
```

In questa parte del programma si introduce il vettore **n_estrazioni**. Questo descrive il numero di estrazioni che bisogna valutare con il Montecarlo, che nel programma è stato stabilito con i dati richiesti della traccia (per una maggiore versatilità potrebbe essere sostituito con un input da tastiera, anche se questo potrebbe portare ad errori legati all'inserimento dei dati manualmente).

Successivamente dopo aver prelevato la lunghezza del vettore **n_estrazioni**, che sarà utile per inizializzare i vari cicli for, si creano tutte le matrici che verranno utilizzate in seguito; questo non è utile ai fini logici del programma, ma permette un'esecuzione più veloce e una migliore gestione della memoria.

FASE 1 attuazione del Montecarlo

```
% --- FASE 1: Generazione e Stima (Montecarlo) ---
% Imposto un ciclo for per i diversi valori di n
for j = 1:num_casistiche
    n = n_estrazioni(j);

    % Pre-alloco la matrice dei venti per il lotto corrente
    v_venti = zeros(n, M);

    for i = 1:M
        %Creo una matrice dove ogni colonna è un'estrazione di n valori
        %Nota:
```

```

        %Assumiamo che weibull_estrazione restituisca
        %un vettore colonna nx1

        v_venti(:, i) = weibull_estrazione(alpha, beta, n);
    end

    % Stima dei parametri alpha e beta per ogni colonna
    for y = 1:M

        % Studio la Carta di Weibull per ogni estrazione del Ciclo
        % Montecarlo
        [x_lin, y_lin] = weibull_carta(v_venti(:, y));

        % Ottengo alpha e beta stimati tramite regressione
        [alpha_stima, beta_stima] = weibull_stima(x_lin, y_lin);

        % Immagazzino i valori nelle matrici
        alpha_matrice(j, y) = alpha_stima;
        beta_matrice(j, y) = beta_stima;
    end
end
end

```

In questa parte di **function** bisogna reiterare il metodo Montecarlo per tutte le varie estrazioni. Questo significa prima costruire una matrice di **M** colonne e **n** righe, dove **n** è il numero di campioni da esaminare, poi bisogna estrarre i risultati della stima di α e β da ogni colonna, in questo caso si chiedeva l'utilizzo di un metodo di stima basato sulla regressione lineare.

Per attuare questo processo è stato necessario concatenare più cicli **for**. Il primo serve per stabilire il numero di campioni da valutare, il secondo costruisce la matrice prima descritta con l'ausilio della **function weibull_estrazione** sopra riportata; questa estrae dati casuali dalla **Weibull** di partenza.

In fine l'ultimo ciclo stima i valori di α e β tramite l'utilizzo delle due funzioni **weibull_carta**, che ricava la **carta di Weibull** dai dati prima estratti e **weibull_stima**, che trova tramite l'interpolazione della carta i valori stimati di α e β .

Tutti i valori stimati vengono salvati in due matrici: ogni riga di queste matrici contiene **M** valori legati di α e β legati all'estrazione di **n** valori, i quali vengono stabiliti nel primo ciclo **for**.

FASE 2 Analisi statistica e grafica

```

% --- FASE 2: Analisi Statistica e Grafica ---

figure('Name', ['Distribuzioni Campionarie Alpha e Beta per ' ...
'M=' num2str(M)]);

for j = 1:num_casistiche
    % Istogramma per i valori stimati di alpha

```

```

subplot(2, 5, j);
histogram(alpha_matrice(j, :), 'Normalization', 'pdf');
title(['Alpha (n = ' num2str(n_estrazioni(j)) ')']);
xlabel('Valore \alpha(m/s)'); ylabel('PDF');
grid on;

% Istogramma per i valori stimati di beta
subplot(2, 5, j + 5);
histogram(beta_matrice(j, :), 'Normalization', 'pdf');
title(['Beta (n = ' num2str(n_estrazioni(j)) ')']);
xlabel('Valore \beta'); ylabel('PDF');
grid on;

% Calcolo valore atteso, mediano e deviazione standard
atteso_Alpha(j) = mean(alpha_matrice(j, :));
mediano_Alpha(j) = median(alpha_matrice(j, :));
dev_Alpha(j) = std(alpha_matrice(j, :));

atteso_Beta(j) = mean(beta_matrice(j, :));
mediano_Beta(j) = median(beta_matrice(j, :));
dev_Beta(j) = std(beta_matrice(j, :));

fprintf('\n--- Statistiche per n=%d ---\n', n_estrazioni(j));
fprintf('Alpha: Atteso=%.4f, Mediano=%.4f, Std=%.4f\n', ...
    atteso_Alpha(j), mediano_Alpha(j), dev_Alpha(j));
fprintf('Beta : Atteso=%.4f, Mediano=%.4f, Std=%.4f\n', ...
    atteso_Beta(j), mediano_Beta(j), dev_Beta(j));

% --- CALCOLO INTERVALLI DI CONFIDENZA 95% ---
% Come richiesto dalla traccia, calcoliamo l'intervallo
% basandoci sui percentili della distribuzione campionaria
% (2.5% e 97.5%)

CI_alpha = prctile(alpha_matrice(j, :), [2.5 97.5]);
fprintf('Intervallo Confidenza 95% Alpha: [%.4f, %.4f]\n', ...
    CI_alpha(1), CI_alpha(2));
end

```

In questa parte si ricavano tutti i valori di natura statistica di α e β e il loro **istogramma**.

Quest'ultimo è una buona stima di quella che è la **densità di probabilità** di una data popolazione.

La prima parte è legata alla creazione degli istogrammi per ogni estrazione.

Anche in questa situazione si è sfruttato un ciclo **for** per analizzare le matrici create nel punto precedente, estraendo così per ogni riga l'istogramma tramite il comando **histogram**. E' stato **normalizzato** come una **densità di probabilità**, questo è esplicitato all'interno degli **input** del comando come '**pdf**'; quindi, l'area sottesa da ogni istogramma è uguale a uno, questa scelta è stata fatta per dare un risultato che permetta una maggiore corrispondenza tra i dati statistici trovati successivamente e l'analisi grafica con gli istogrammi.

La seconda parte si basa sul trovare tutti i vari dati richiesti dalla traccia, implementando diverse funzioni già presenti in **MATLAB**.

Per il calcolo degli intervalli di confidenza di α si è utilizzato la funzione “**prctile**”, che richiede in **input** i dati da analizzare e i percentili, successivamente riordinando i dati e calcolano l’intervallo di confidenza desiderato basandosi sui percentili forniti, in questo caso del **95%**. Questa risulta essere particolarmente versatile, poiché basandosi sui percentili riesce a trovare gli intervalli di confidenza in riferimento sui dati forniti e non sui parametri; di conseguenza posso fornirli come **input** a un qualsiasi tipo di distribuzione.

Fase 3 calcolo della potenza elettrica

```
% --- FASE 3: Calcolo Potenza Elettrica ---
rho = 1.225; % densità dell'aria in kg/m^3
A = 40;      % area massa eolica [m^2]

% Pre-allocazione vettore potenza
potenzaElettrica = zeros(1, num_casistiche);

fprintf('\n--- Analisi Potenza Elettrica ---\n');
for j = 1:num_casistiche
    % La traccia richiede di usare "valore alfa mediano" come velocità v
    % P = 0.5 * rho * A * v^3
    potenzaElettrica(j) = 0.5 * rho * A * (mediano_Alpha(j).^3);

    fprintf('Potenza per n=%d: %.2f W\n', n_estrazioni(j), ...
            potenzaElettrica(j));
end
```

In questa parte si calcolano le varie potenze in funzione dei valori mediani di α prima calcolati; questo viene fatto per ogni distribuzione basata su **n** estrazioni.

Fase 4 verifica dello scarto percentuale e calcolo dell’output della function

```
% --- FASE 4: Verifica Scarto Percentuale ---
% Si valuta il numero ottimale per scarto < 15%
limite = 15;

fprintf('\n--- Analisi Scarto Percentuale (Limite %d%%) ---\n',limite);

n_minimo = [];
idx_minimo = 1; % Indice per riempire il vettore n_minimo

for j = 1:num_casistiche
    % Calcolo scarto percentuale assoluto rispetto ai valori veri
    % (input alpha, beta)
    scartoAlpha = (abs(atteso_Alpha(j) - alpha) / alpha) * 100;
    scartoBeta  = (abs(atteso_Beta(j) - beta) / beta) * 100;

    fprintf('n=%d -> Scarto Alpha: %.2f%%, Scarto Beta: %.2f%%\n', ...
            n_estrazioni(j), scartoAlpha, scartoBeta);

    if scartoAlpha <= limite && scartoBeta <= limite
```

```

        n_minimo(idx_minimo) = n_estrazioni(j);
        idx_minimo = idx_minimo + 1;

    end
end

% --- CONCLUSIONI FINALI ---
fprintf('\n-----\n');
if isempty(n_minimo)
    fprintf(['NESSUN campione soddisfa il requisito del 15% su ' ...
            'entrambi i parametri.\n']);
    n_ottimale = NaN; % Nessun valore trovato
else
    % Poiché n_estrazioni è ordinato (3, 5, 10...), il primo valore
    % trovato è automaticamente il minimo.
    % Ma per sicurezza usiamo min().
    n_ottimale = min(n_minimo);

    fprintf('Campioni che soddisfano il requisito (<15%): %s\n', ...
            num2str(n_minimo));
    fprintf(['>>> IL NUMERO MINIMO DI CAMPIONI CONSIGLIATO È:' ...
            'n = %d <<<\n'], n_ottimale);
end
fprintf('-----\n');
end

```

In questa parte si valutano quali risultati sono al disotto di uno scarto del **15%** rispetto ai valori reali di α e β per poi scegliere il minor numero di estrazioni che rispetta queste caratteristiche.

Per fare ciò è stata introdotta una nuova variabile **n_minimo**.

Quest'ultima rappresenta un vettore che conterrà i valori di **n** che rispettano i limiti descritti in precedenza, venendo aggiornata ad ogni ciclo, ma al contrario di tutte le altre variabili utilizzate precedentemente non abbiamo un dato che ci permette di sapere con certezza la dimensione finale del vettore, e dunque non può essere inizializzata come le altre (ossia come una matrice di zeri che successivamente viene aggiornata).

Si è quindi scelto di inizializzare la variabile come una matrice vuota, che viene riempita solo quando il valore rispetta i limiti preposti, tramite l'ausilio della variabile **idx_minimo**, che serve solo a posizionare correttamente i valori nel vettore.

Una volta esaurito il ciclo che verifica per quali valori di **n** lo scarto percentuale è inferiore al **15%** (sia per α che per β), si passa alla parte finale del programma che controlla se la matrice **n_minimo** contiene degli elementi (di fatto col metodo scelto se nessun valore di **n** rispetta le caratteristiche la variabile rimane una matrice vuota), per poi selezionare, se presente, il valore più piccolo di quelli ricavati.

3.2 Function 1: Estrazione Campioni

Genera n valori casuali di **probabilità** F e ricava le corrispondenti velocità del **vento** v invertendo la formula di Weibull, simulando così l'acquisizione dati.

```
function v_venti = weibull_estrazione(n, alpha, beta)
    %Genera campioni di velocità del vento.

    %Questa funzione utilizza il metodo della "Trasformata Inversa" per
    % generare n valori che seguono la distribuzione di Weibull.

    %INPUT:
    %n - Numero di campioni da generare
    %alpha - Parametro di scala (dato dal problema)
    %beta - Parametro di forma (dato dal problema)

    %OUTPUT:
    %v_venti - Vettore colonna contenente le velocità generate [m/s]

    %Generazione delle probabilità cumulate casuali:

    %rand crea numeri tra 0 e 1 (rappresentano la F nella CDF)
    % (CDF = Cumulative Density Function)

    F = rand(n, 1);

    %Inversione della formula della CDF di Weibull
    %Dalla teoria:  $F = 1 - \exp(-(v/\alpha)^\beta)$ 

    %Invertendo per v otteniamo:  $v = \alpha * (-\ln(1-F))^{1/\beta}$ 
    % Nota: in Matlab log() è il logaritmo naturale (ln)

    v_venti = alpha .* (-log(1 - F)).^(1/beta);
    %resituisce in output un vettore colonna

    v_venti = v_venti(:);

end
```

3.3 Function 2: Creazione carta di Weibull

Riceve in **input** le velocità, le **ordina** in senso crescente e calcola le coordinate (x, y) per il piano di **Weibull**, applicando lo **stimatore di Blom**.

```
function [x, y] = weibull_carta(v_venti)
    % Calcola i punti per la carta di Weibull

    % Input:
    % X -> vettore colonna del campione
    % Output:
    % Xcarta -> log(X ordinato)
    % Ycarta -> log(-log(1-F(X))) per carta di Weibull
```



```

%Ordina il campione (da esperimenti più piccoli a più grandi)
Xs = sort(X);

%Dimensione del campione
n = length(Xs);

%Calcola F(X) con formula stimatore di Bloom della funzione di
% distribuzione per ogni esperimento i_esimo del campione Xs
i=(1:n)';
F_Blom = (i - 0.5) ./ (n + 0.25); % vettore colonna

%linearizzazione per la carta di Weibull e per poter
%applicare regressione lineare e trovare stimatori alfa e beta
Ycarta = log(-log(1 - F_Blom));

% Asse X della carta: log(X)
Xcarta = log(Xs);

end

```

3.4 Function 3: Stima Parametri

Effettua la **regressione lineare** sui punti (x, y) per ricavare i parametri stimati $\hat{\alpha}$ e $\hat{\beta}$ e dell'iterazione corrente.

```

function [alpha_stima, beta_stima] = weibull_stima(x, y)
% Calcolo i coefficienti della retta di regressione lineare che possiede
% il seguente andamento:

%                               y = beta * x + intercetta

% dove
%                               intercetta = - beta * ln(alpha)

% Adopero il metodo ai minimi quadrati

% vettore dei coefficienti del polinomio di grado 1:
p = polyfit(x, y, 1);

% valore della pendenza della retta di regressione:
beta_stima = p(1);

% valore di passaggio della retta di regressione per l'asse delle ordinate
intercetta = p(2);

% valore stimato del parametro alpha:
alpha_stima = exp(-intercetta./beta_stima);

end

```

4. Analisi dei Risultati

4.1 Distribuzioni Campionarie:

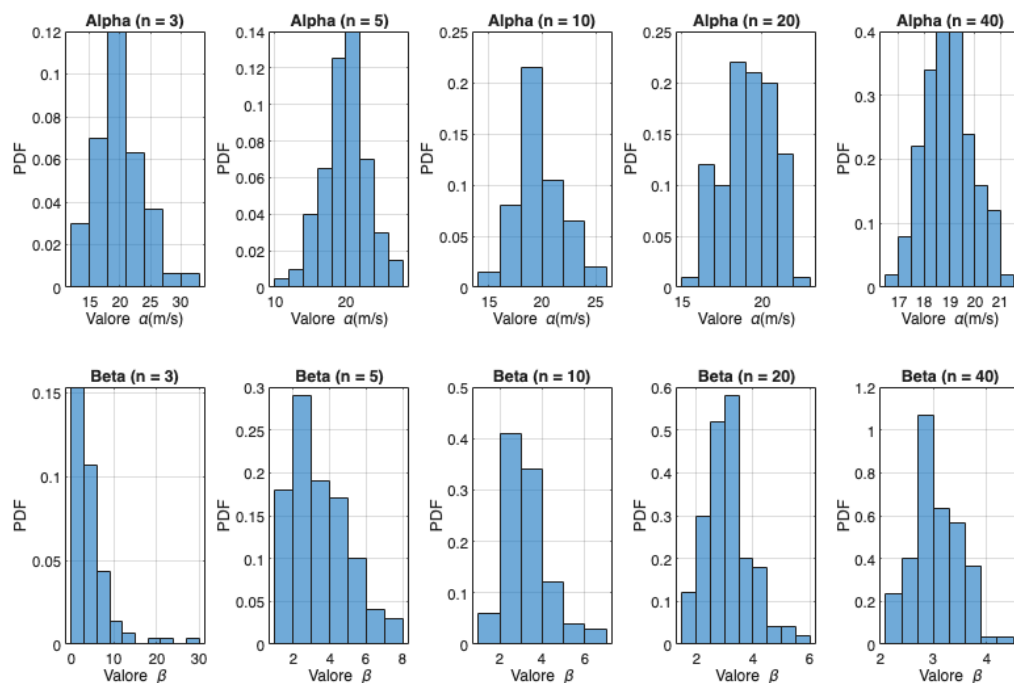
Di seguito si riportano gli **istogrammi** delle stime di α e β al variare di n . Si nota come, all'aumentare della numerosità del campione, la **dispersione diminuisca**, concentrandosi attorno ai **valori veri**.

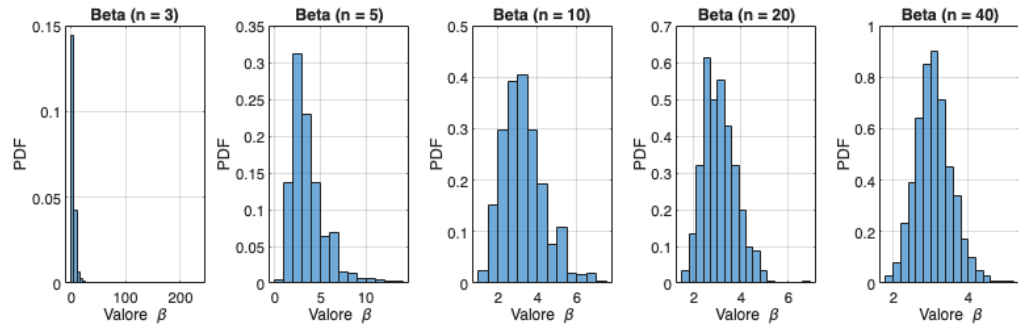
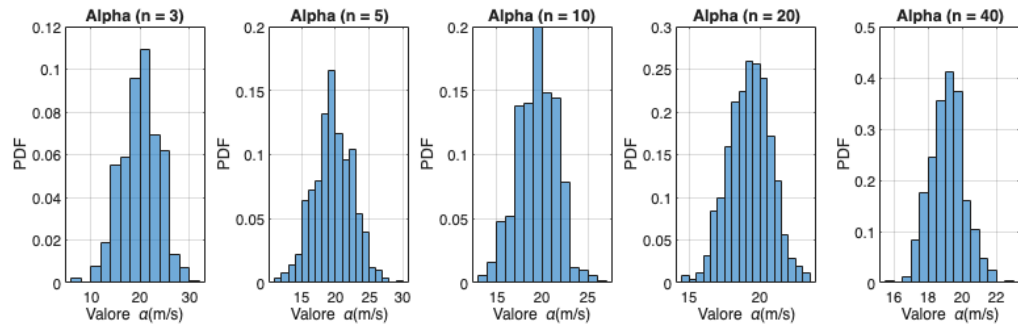
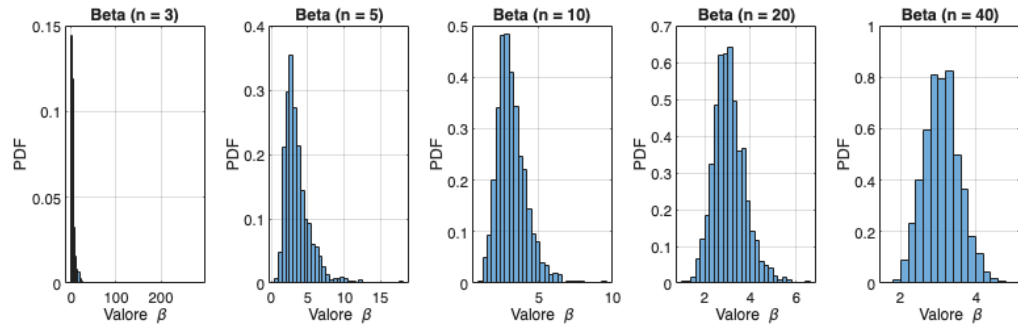
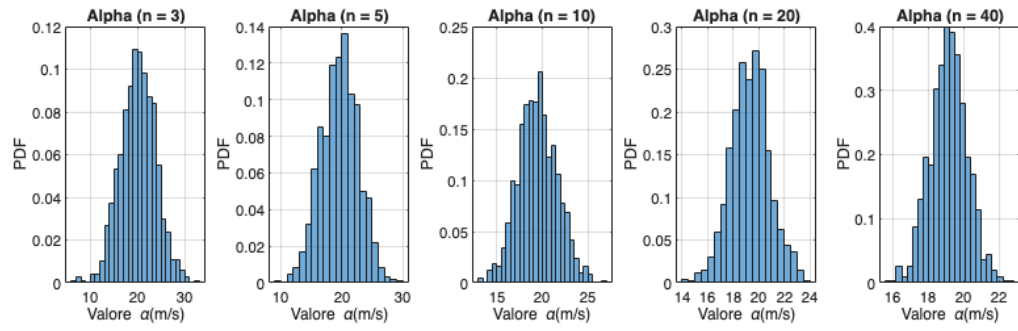
Commenti:

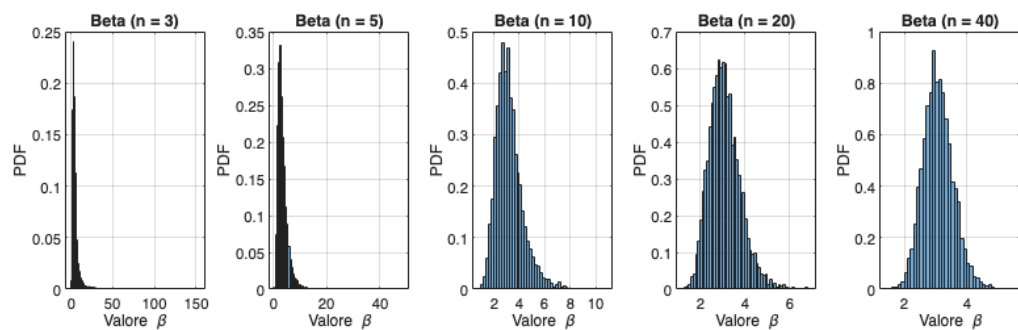
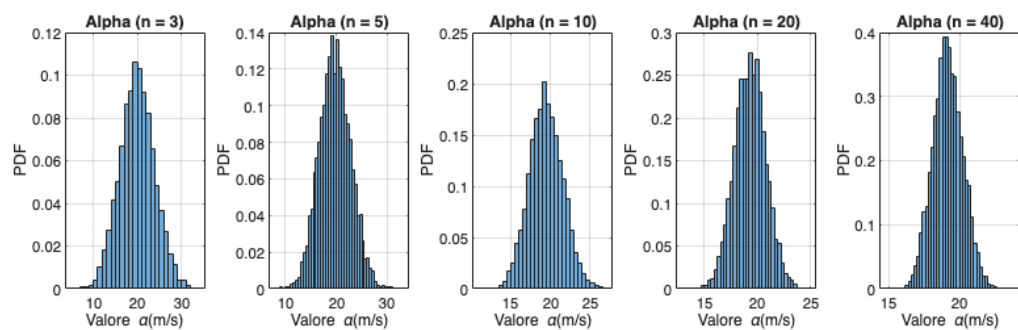
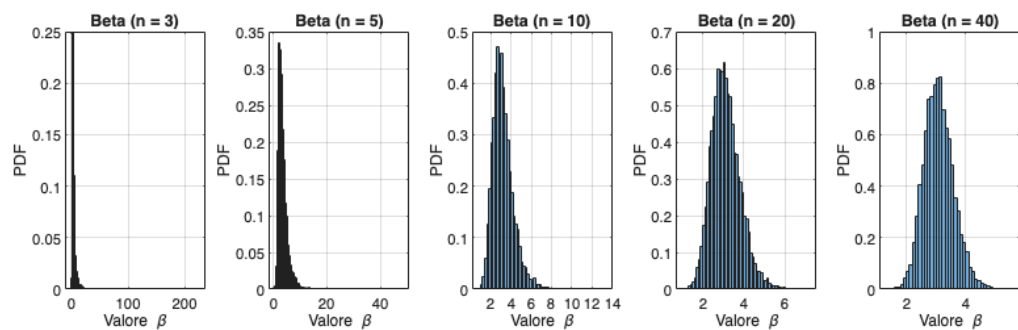
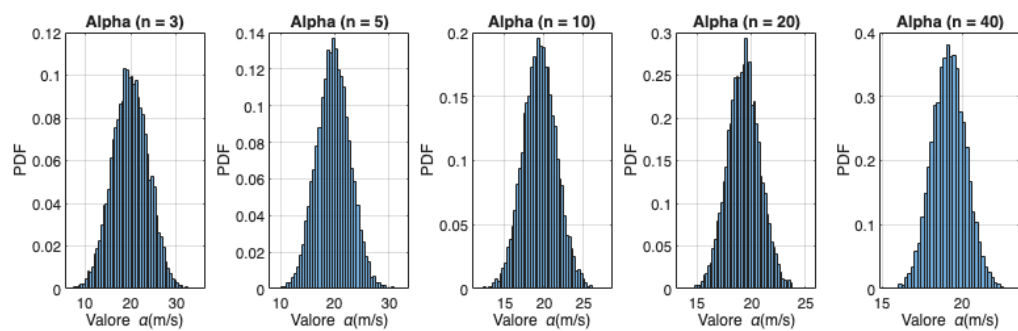
Dall'analisi degli istogrammi e dei parametri statistici, emerge chiaramente l'effetto della legge dei grandi numeri. Per $n = 3$, la curva è estremamente piatta e larga (alta dispersione), rendendo la stima quasi casuale. Passando a $n = 40$, la distribuzione si stringe notevolmente attorno al valore atteso. Numericamente, questo è confermato dalla **Deviazione Standard di Alpha**, che crolla da circa **3.9** (con $n = 3$) a circa **1.0** (con $n = 40$) nelle simulazioni con M elevato.

Istogrammi (in ordine):

- **M=100:**
- **M=500:**
- **M=1.000:**
- **M=5.000:**
- **M=10.000:**







4.2 Analisi Statistica

La tabella seguente riassume i parametri statistici calcolati su **M=10.000** iterazioni del Montecarlo.

Numerosità Campione (n)	Scarto Medio su α	Scarto Medio su β	Esito (Target $\leq 15\%$)
3	4.59%	57.99%	NON ACCETTABILE
5	3.80%	15.13%	NON ACCETTABILE*
10	2.55%	3.71%	OTTIMALE (Minimo)
20	1.56%	0.49%	ACCETTABILE
40	0.92%	0.42%	ACCETTABILE

**Nota: Con $n = 5$, lo scarto su β supera, seppur di poco, la soglia limite del 15%.*

Commenti:

Osservando i risultati per **M=10.000**, si nota una differenza sostanziale nella capacità di stimare α (scala) rispetto a β (forma):

- Il parametro α viene stimato discretamente bene anche con pochi campioni (errore $< 5\%$ già con $n = 3$).
- Il parametro β è estremamente sensibile: con $n = 3$, lo scarto percentuale è del **57%** circa, rendendo la stima inutilizzabile. Con $n = 5$, lo scarto scende ma risulta essere ancora di poco superiore al **15%**.

4.3 Stima della Potenza Eolica

Utilizzando la formula $P = \frac{1}{2} \rho A v^3$ (con ρ densità dell'aria, $A = 40m^2$ superficie pala eolica e v velocità del vento, con v pari al valore mediano di α stimato), si è calcolata la potenza producibile per ogni lotto.

Commenti:

Un dato critico emerge dal calcolo della Potenza Stimata. Con $n = 3$, la potenza calcolata è spesso sovrastimata (circa **191 kW** nel caso **M=10.000**) rispetto al valore che si stabilizza con campioni numerosi (circa **172 kW** con $n = 40$).

Utilizzare pochi campioni non solo porta a incertezza statistica, ma in questo caso specifico rischia di sovrastimare il potenziale energetico del sito, portando a investimenti economici errati basati su previsioni troppo ottimistiche.

4.4 Analisi di Sensibilità (Parametro M)

Le simulazioni sono state ripetute variando il numero di iterazioni Montecarlo **M** da 100 a 10.000.

Si è osservato che con **M=100** (poche iterazioni), i risultati sono instabili. Ad esempio, in una delle esecuzioni a basso M, il caso **n = 5** sembrava accettabile o molto vicino al limite. Aumentando M a **5.000** e **10.000**, i valori statistici convergono e si stabilizzano. Questo dimostra che per validare un modello statistico di questo tipo sono necessarie almeno 5.000 iterazioni per eliminare il 'rumore' numerico e avere intervalli di confidenza affidabili.

5. Conclusioni

Confrontando lo scarto percentuale medio tra i valori stimati e quelli veri ($\alpha = 19, \beta = 3.1$), si è determinato che il numero minimo di campioni per mantenere l'errore al di sotto del 15% su entrambi i parametri è **n = {10}**.

Commenti:

Sulla base dei dati per **M=10.000**:

- **n = 3**: Scarto Beta ~57% (INACCETTABILE)
- **n = 5**: Scarto Beta ~16.2% (NON ACCETTABILE, > 15%)
- **n = 10**: Scarto Beta ~3.9% (ACCETTABILE, < 15%)

Dall'analisi degli scarti percentuali medi, si determina che il numero minimo di campioni necessario è **n = 10**. Sebbene il parametro α sia stabile già con 5 misurazioni, il parametro β (che influenza la forma della curva e quindi la distribuzione delle frequenze di vento) richiede un set di dati maggiore per scendere sotto la soglia di errore del 15%.

Questo suggerisce che installare un anemometro per un periodo troppo breve (pochi campioni) porta a stime inaffidabili del potenziale eolico.

[Appendice: Codice MATLAB]

Script Principale:

```
clear; clc; close all;

% Parametri
alpha_vero = 19;
beta_vero = 3.1;
ciclo_M = [100, 500, 1000, 5000, 10000]; % Analisi di sensibilità - 5 iterazioni

% Ciclo per l'analisi di sensibilità
for M = ciclo_M
    fprintf('Esecuzione con M = %d...\n', M);
    esame(alpha_vero, beta_vero, M);
    pause(1); % Pausa per visualizzare i grafici
end
```

